

LAB REPORT

Counters

3220102382 潘谷雨

2024.12.5

1. Problem Description

(1) 16-bit synchronous counter

Consider the circuit in Figure 1. It is a 4-bit synchronous counter which uses four T-type flip-flops. The counter increments its count on each positive edge of the clock if the Enable signal is asserted. The counter is reset to 0 by using the Reset signal. You are to implement a 16-bit counter of this type.

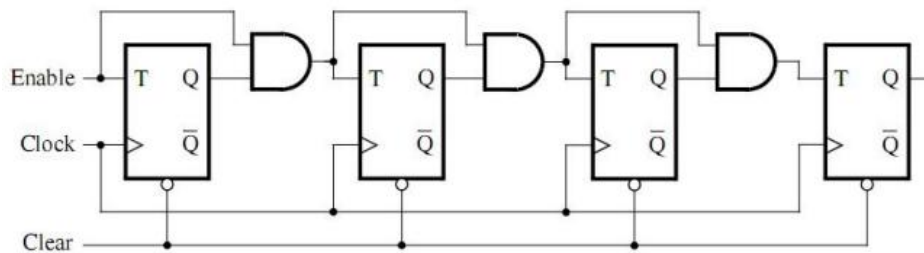


Figure 1. A 4-bit counter.

1. Write a VHDL file that defines a 16-bit counter by using the structure depicted in Figure 1, and compile the circuit.
2. Simulate your circuit to verify its correctness.
3. Augment your VHDL file to use the pushbutton KEY0 as the Clock input, switches SW1 and SW0 as Enable and Reset inputs, and 7-segment displays HEX3-0 to display the hexadecimal count as your circuit operates. Make the necessary pin assignments and compile the circuit.
4. Implement your circuit on the DE2 board and test its functionality by operating the implemented switches.
5. Implement a 4-bit version of your circuit and use the Quartus II RTL Viewer to see how Quartus II software synthesized your circuit.

(2) running word

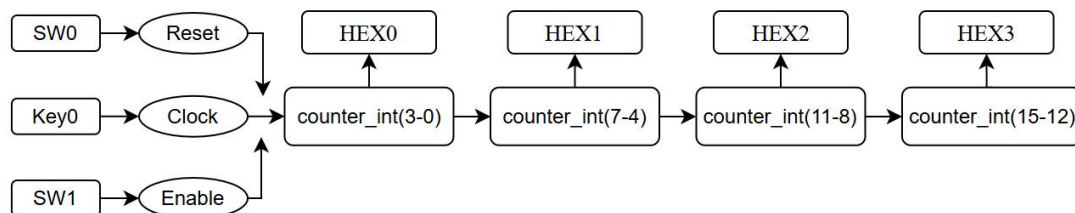
Design and implement a circuit that displays the word HELLO, in ticker tape fashion, on the eight 7-segment displays HEX7-0. Make the letters move from right to left in intervals of about one second. The patterns that should be displayed in successive clock intervals are given in Table 1.

Clock cycle	Displayed pattern				
0			H	E	L L O
1			H	E	L L O
2		H	E	L L	O
3	H	E	L L	O	
4	E	L L	O		H
5	L L	O			H E
6	L O			H E	L
7	O			H E L L	
8			H E L L	O	
...	and so on				

Table 1. Scrolling the word HELLO in ticker-tape fashion.

2. Design Formulation

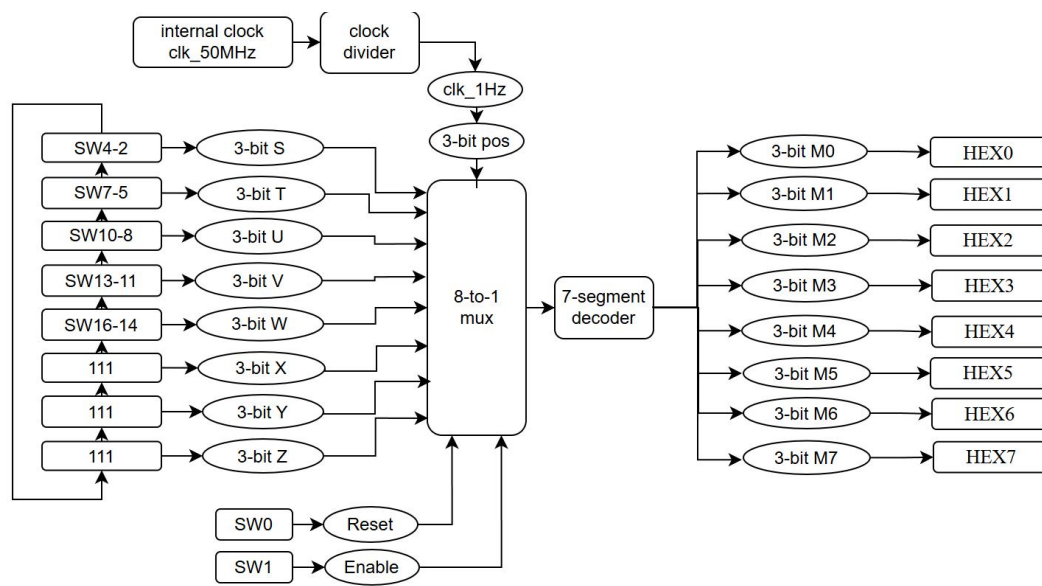
(1)16-bit synchronous counter



Pic.1.1 design formulation of 16-bit synchronous counter

Use 4 T flip-flops to implement a 4-bit counter like the circuit Pic.1.1 shows. 4 4-bit counters form a 16-bit counter and each 4-bit output corresponds to the HEX0-3.

(2)running word



Pic.2.1 design formulation of running word

Use an 8-to-1 multiplexer to select a character for display on one digital tube and all eight digital tubes demonstrate letters in sequence. In circuit in Pic.2.1, use a clock signal to control the

multiplexer selection. When the clock is enabled, the multiplexer selection will remain unchanged until the next clock rising edge.

3. Design Entry

(1) 16-bit synchronous counter

seven_segment_decoder.vhd:

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity seven_segment_decoder is
6  Port ( bin : in  STD_LOGIC_VECTOR(3 downto 0);
7        seg : out STD_LOGIC_VECTOR(6 downto 0));
8  end seven_segment_decoder;
9
10 architecture Behavioral of seven_segment_decoder is
11 begin
12     process(bin)
13     begin
14         case bin is
15             when "0000" => seg <= "1000000"; -- 0
16             when "0001" => seg <= "1111001"; -- 1
17             when "0010" => seg <= "0100100"; -- 2
18             when "0011" => seg <= "0110000"; -- 3
19             when "0100" => seg <= "0011001"; -- 4
20             when "0101" => seg <= "0010010"; -- 5
21             when "0110" => seg <= "0000010"; -- 6
22             when "0111" => seg <= "1111000"; -- 7
23             when "1000" => seg <= "0000000"; -- 8
24             when "1001" => seg <= "0010000"; -- 9
25             when "1010" => seg <= "0001000"; -- A
26             when "1011" => seg <= "0000011"; -- B
27             when "1100" => seg <= "1000110"; -- C
28             when "1101" => seg <= "1000010"; -- D
29             when "1110" => seg <= "0000110"; -- E
30             when others => seg <= "0001110"; -- F
31         end case;
32     end process;
33 end Behavioral;
```

Sixteen_bit_synchronous_counter.vhd:

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity sixteen_bit_synchronous_counter is
6  Port ( Clock : in  STD_LOGIC;
7        Enable : in  STD_LOGIC;
8        Reset : in  STD_LOGIC;
9        Count : buffer STD_LOGIC_VECTOR(15 downto 0);
10        HEX0,HEX1,HEX2,HEX3: out STD_LOGIC_VECTOR(6 downto 0));
11 end sixteen_bit_synchronous_counter;
```

```

13 architecture Behavioral of sixteen_bit_synchronous_counter is
14     signal count_int : unsigned(15 downto 0) := (others => '0');
15     component seven_segment_decoder
16     Port ( bin : in  STD_LOGIC_VECTOR(3 downto 0);
17           seg : out STD_LOGIC_VECTOR(6 downto 0));
18     end component;
19 begin
20     process(Clock, Reset)
21     begin
22         if Reset = '1' then
23             count_int <= (others => '0');
24         elsif rising_edge(Clock) then
25             if Enable = '1' then
26                 count_int <= count_int + 1;
27             end if;
28         end if;
29     end process;
31     Count <= std_logic_vector(count_int);
32
33     M_HEX3: seven_segment_decoder port map (std_logic_vector(count_int(15 downto 12)), HEX3);
34     M_HEX2: seven_segment_decoder port map (std_logic_vector(count_int(11 downto 8)), HEX2);
35     M_HEX1: seven_segment_decoder port map (std_logic_vector(count_int(7 downto 4)), HEX1);
36     M_HEX0: seven_segment_decoder port map (std_logic_vector(count_int(3 downto 0)), HEX0);
37 end Behavioral;

```

(2)running word

eight_to_one_mux.vhd:

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity eight_to_one_mux is
5  Port ( C: in  STD_LOGIC_VECTOR(2 downto 0); -- Selection lines
6        S, T, U, V, W, X, Y, Z : in  STD_LOGIC_VECTOR (2 downto 0);
7        M : out STD_LOGIC_VECTOR (2 downto 0)); -- Output data
8  end eight_to_one_mux;
9
10 architecture Behavioral of eight_to_one_mux is
11 begin
12     process(C, S, T, U, V, W, X, Y, Z)
13     begin
14         case C is
15             when "000" => M <= S;
16             when "001" => M <= T;
17             when "010" => M <= U;
18             when "011" => M <= V;
19             when "100" => M <= W;
20             when "101" => M <= X;
21             when "110" => M <= Y;
22             when "111" => M <= Z;
23             when others=> M <= "111";
24         end case;
25     end process;
26 end Behavioral;

```

clock_divider.vhd:

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity clock_divider is
6  Port ( clk_50MHz : in  STD_LOGIC;
7        reset : in  STD_LOGIC;
8        clk_1Hz : out  STD_LOGIC);
9  end clock_divider;
10
11 architecture Behavioral of clock_divider is
12     signal count : unsigned(25 downto 0) := (others => '0');
13 begin
14     process(clk_50MHz, reset)
15     begin
16         if reset = '1' then
17             count <= (others => '0');
18             clk_1Hz <= '0';
19         elsif rising_edge(clk_50MHz) then
20             if count = 50000000 - 1 then--if count = 100 - 1 then--
21                 count <= (others => '0');
22                 clk_1Hz <= '1';
23             else
24                 count <= count + 1;
25                 if count = 250000000 - 1 then--if count = 50 - 1 then
26                     clk_1Hz <= '0';
27                 end if;
28             end if;
29         end if;
30     end process;
31 end Behavioral;

```

seven_segment_decoder.vhd:

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity seven_segment_decoder is
6  Port ( bin : in  STD_LOGIC_VECTOR(2 downto 0);
7        seg : out  STD_LOGIC_VECTOR(6 downto 0));
8  end seven_segment_decoder;
9
10 architecture Behavioral of seven_segment_decoder is
11 begin
12     process(bin)
13     begin
14         case bin is
15             when "000" => seg <= "0001001"; -- H
16             when "001" => seg <= "0000110"; -- E
17             when "010" => seg <= "1000111"; -- L
18             when "011" => seg <= "1000000"; -- O
19             when others => seg <= "1111111"; -- black
20         end case;
21     end process;
22 end Behavioral;

```

running_word.vhd:

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity running_word is
6  Port ( clk : in  STD_LOGIC;
7         Reset : in  STD_LOGIC;
8         Enable : in  STD_LOGIC;
9         SW : in STD_LOGIC_VECTOR(14 downto 0);
10        clk_1Hz : buffer STD_LOGIC;
11        M0, M1, M2, M3, M4, M5, M6, M7: buffer STD_LOGIC_VECTOR(2 DOWNTO 0);
12        shift_reg: buffer STD_LOGIC_VECTOR(23 downto 0);
13        pos: buffer STD_LOGIC_VECTOR(2 DOWNTO 0);
14        HEX0, HEX1, HEX2, HEX3, HEX4, HEX5, HEX6, HEX7: out STD_LOGIC_VECTOR(6 downto 0));
15  end running_word;
16
17  architecture Behavioral of running_word is
18  component seven_segment_decoder
19  Port ( bin : in  STD_LOGIC_VECTOR(2 downto 0);
20        seg : out  STD_LOGIC_VECTOR(6 downto 0));
21  end component;
22
23  component clock_divider
24  Port ( clk_50MHz : in  STD_LOGIC;
25        reset : in  STD_LOGIC;
26        clk_1Hz : out  STD_LOGIC);
27  end component;
28
29  COMPONENT eight_to_one_mux
30  PORT (C, S, T, U, V, W, X, Y, Z : IN STD_LOGIC_VECTOR(2 DOWNTO 0);
31        M : OUT STD_LOGIC_VECTOR(2 DOWNTO 0));
32  END COMPONENT;
33
34  begin
35  -- 时钟分频器实例
36  CLK_DIV: clock_divider port map (clk, Reset, clk_1Hz);
37
38  -- 更新位置
39  process(Reset, Enable, clk_1Hz)
40  begin
41  if reset = '1' then
42  pos <= "000";
43  elsif Enable = '1' and rising_edge(clk_1Hz) then
44  if pos = "000" then
45  pos <= "111"; -- 重置位置
46  else
47  pos <= std_logic_vector(unsigned(pos) - 1);
48  end if;
49  end if;
50  end process;

```

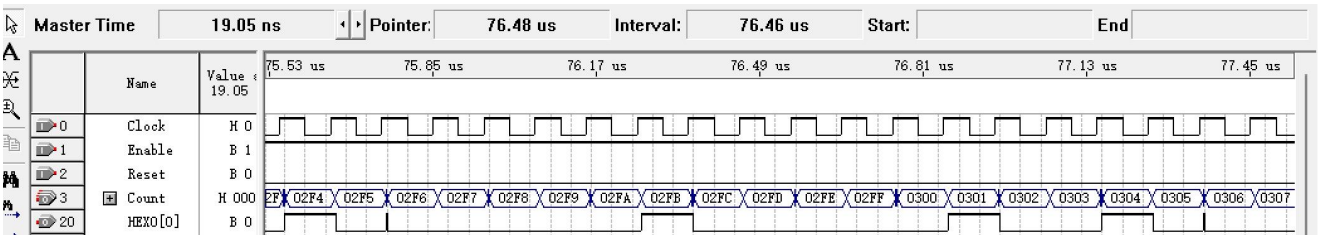
```
shift_reg <= "11111111" & SW(14 DOWNTO 0);
-- 对应位置
MUX0: eight_to_one_mux PORT MAP (pos, shift_reg(2 DOWNTO 0), shift_reg(5 DOWNTO 3), shift_reg(8 DOWNTO 6), shift_reg(11 DOWNTO 9),
    shift_reg(14 DOWNTO 12), shift_reg(17 DOWNTO 15), shift_reg(20 DOWNTO 18), shift_reg(23 DOWNTO 21), M0);
MUX1: eight_to_one_mux PORT MAP (pos, shift_reg(5 DOWNTO 3), shift_reg(8 DOWNTO 6), shift_reg(11 DOWNTO 9), shift_reg(14 DOWNTO 12),
    shift_reg(17 DOWNTO 15), shift_reg(20 DOWNTO 18), shift_reg(23 DOWNTO 21), shift_reg(2 DOWNTO 0), M1);
MUX2: eight_to_one_mux PORT MAP (pos, shift_reg(8 DOWNTO 6), shift_reg(11 DOWNTO 9), shift_reg(14 DOWNTO 12), shift_reg(17 DOWNTO 15),
    shift_reg(20 DOWNTO 18), shift_reg(23 DOWNTO 21), shift_reg(2 DOWNTO 0), shift_reg(5 DOWNTO 3), M2);
MUX3: eight_to_one_mux PORT MAP (pos, shift_reg(11 DOWNTO 9), shift_reg(14 DOWNTO 12), shift_reg(17 DOWNTO 15), shift_reg(20 DOWNTO 18),
    shift_reg(23 DOWNTO 21), shift_reg(2 DOWNTO 0), shift_reg(5 DOWNTO 3), shift_reg(8 DOWNTO 6), M3);
MUX4: eight_to_one_mux PORT MAP (pos, shift_reg(14 DOWNTO 12), shift_reg(17 DOWNTO 15), shift_reg(20 DOWNTO 18), shift_reg(23 DOWNTO 21),
    shift_reg(2 DOWNTO 0), shift_reg(5 DOWNTO 3), shift_reg(8 DOWNTO 6), shift_reg(11 DOWNTO 9), M4);
MUX5: eight_to_one_mux PORT MAP (pos, shift_reg(17 DOWNTO 15), shift_reg(20 DOWNTO 18), shift_reg(23 DOWNTO 21), shift_reg(2 DOWNTO 0),
    shift_reg(5 DOWNTO 3), shift_reg(8 DOWNTO 6), shift_reg(11 DOWNTO 9), shift_reg(14 DOWNTO 12), M5);
MUX6: eight_to_one_mux PORT MAP (pos, shift_reg(20 DOWNTO 18), shift_reg(23 DOWNTO 21), shift_reg(2 DOWNTO 0), shift_reg(5 DOWNTO 3),
    shift_reg(8 DOWNTO 6), shift_reg(11 DOWNTO 9), shift_reg(14 DOWNTO 12), shift_reg(17 DOWNTO 15), M6);
MUX7: eight_to_one_mux PORT MAP (pos, shift_reg(23 DOWNTO 21), shift_reg(2 DOWNTO 0), shift_reg(5 DOWNTO 3), shift_reg(8 DOWNTO 6),
    shift_reg(11 DOWNTO 9), shift_reg(14 DOWNTO 12), shift_reg(17 DOWNTO 15), shift_reg(20 DOWNTO 18), M7);

71      --解码输出
72      CHAR0: seven_segment_decoder PORT MAP (M0, HEX0);
73      CHAR1: seven_segment_decoder PORT MAP (M1, HEX1);
74      CHAR2: seven_segment_decoder PORT MAP (M2, HEX2);
75      CHAR3: seven_segment_decoder PORT MAP (M3, HEX3);
76      CHAR4: seven_segment_decoder PORT MAP (M4, HEX4);
77      CHAR5: seven_segment_decoder PORT MAP (M5, HEX5);
78      CHAR6: seven_segment_decoder PORT MAP (M6, HEX6);
79      CHAR7: seven_segment_decoder PORT MAP (M7, HEX7);
80      end Behavioral;
```

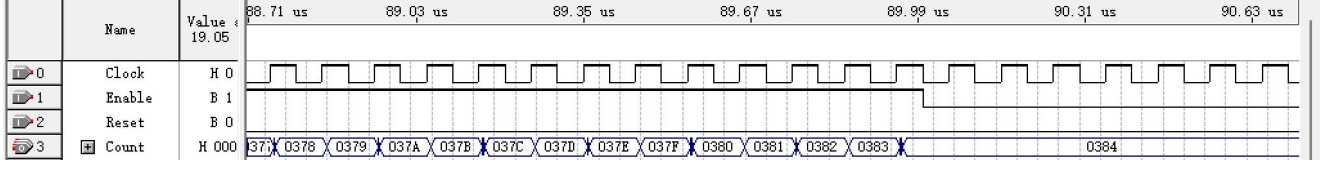
4. Simulation and Synthesis Results

(1)16-bit synchronous counter

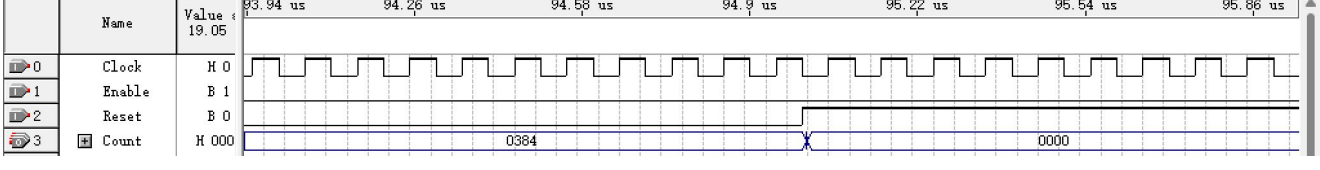
Clock on the run:



Enable on effect:

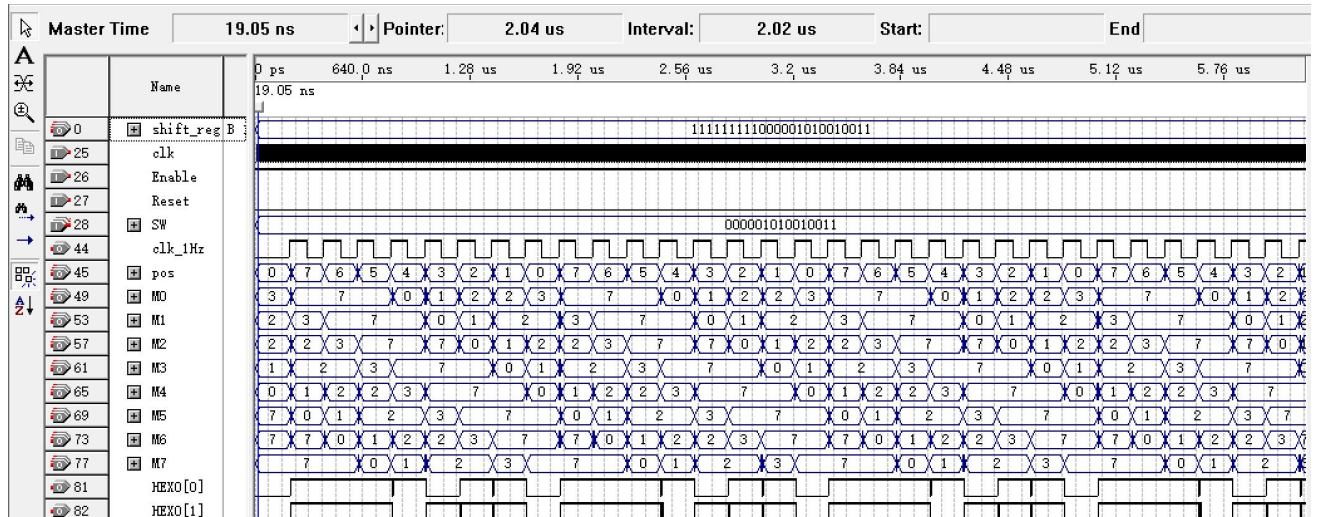


Reset on effect:

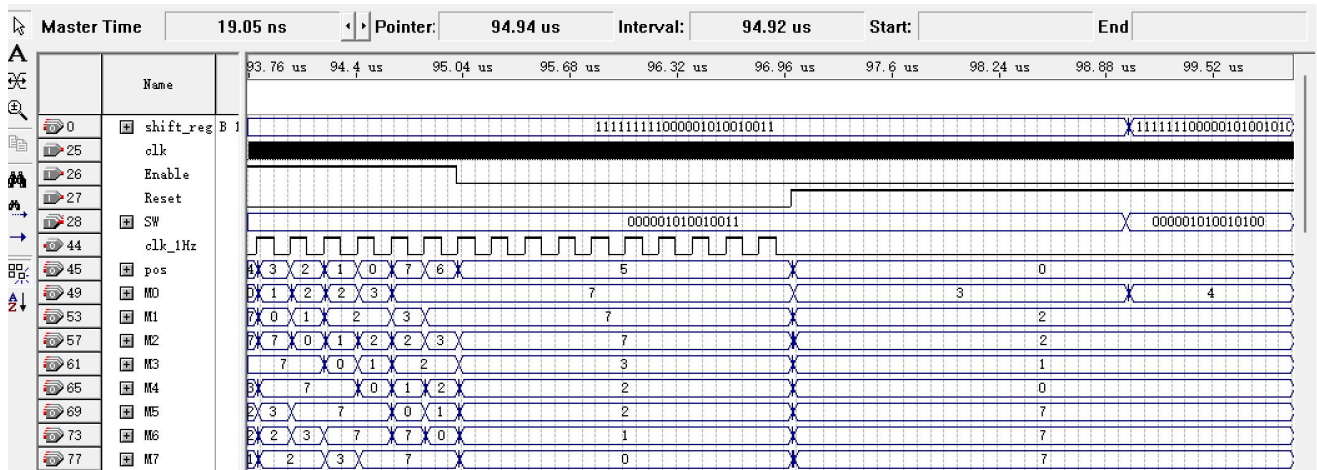


(2)running word

Clock on the run:



Enable and Reset on effect:

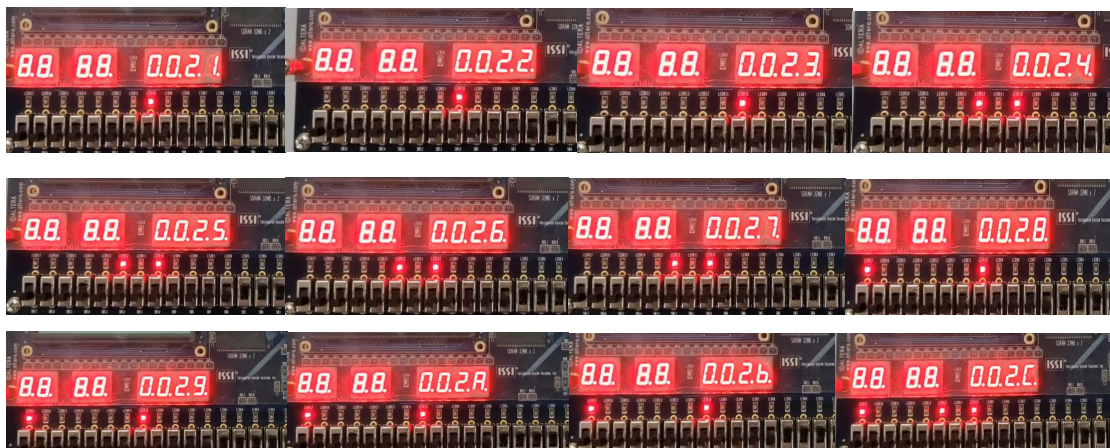


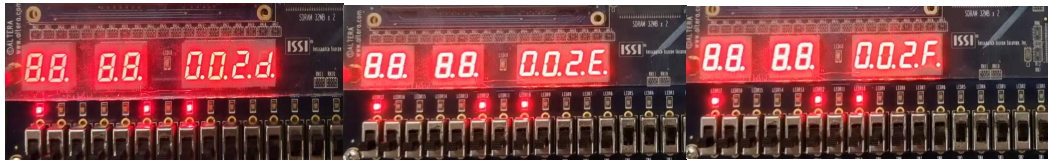
5. Experimental Results

(1) and (2) are proved in physical experiment.

(1)16-bit synchronous counter

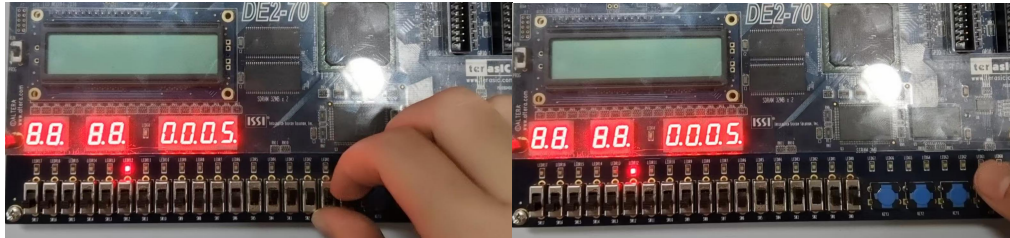
When “Reset” is assigned as “0” and “Enable” as “1”, press key0 and the digital tube demonstrates adding numbers like Pic.1.2 displays.





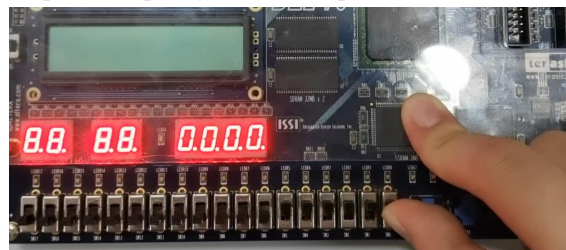
Pic.1.2 16-bit synchronous counter(enabled)

When “Reset” is assigned as “0” and “Enable” as “0”, press key0 and number on the digital tube does not change as Pic.1.3 displays.



Pic.1.3 16-bit synchronous counter(disabled)

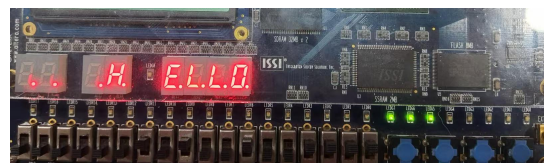
When Reset switch is pushed up, digital tube displays number “0000” as Pic.1.4.



Pic.1.4 16-bit synchronous counter(reset)

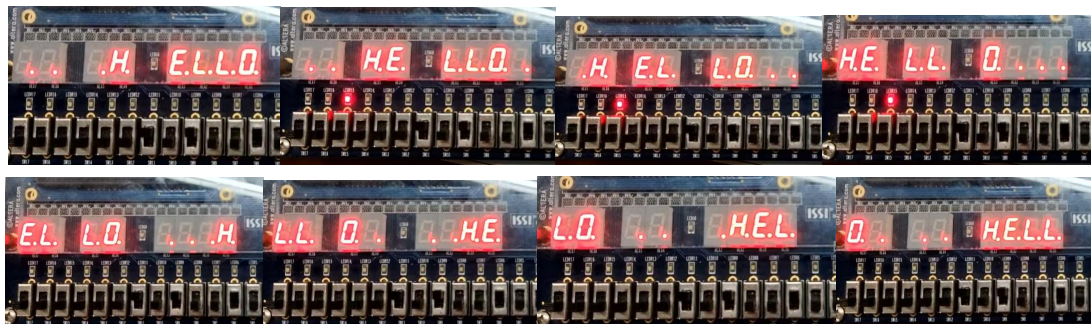
(2)running word

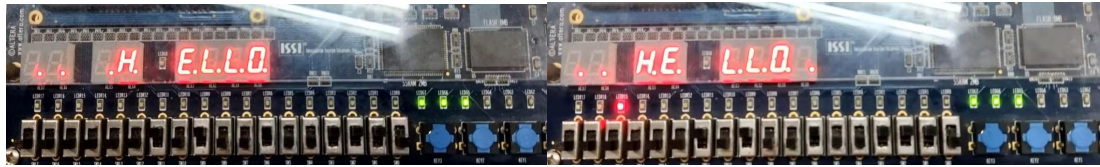
When “Reset” is assigned as “0”, the digital tube demonstrates initial pattern like Pic.2.2 displays.



Pic.2.2 running word(reset)

Keep “Enable” as “1”. When Reset switch is pushed up, digital tube displays running word loop as Pic.2.3 displays.





Pic.2.3 running word(enabled)

Keep “Reset” as “0”. When Enable switch is pushed down, running word loop stops as Pic.2.4 displays.



Pic.2.4 running word(disabled)

6. Discussion and Conclusion

Initially I planned to use two-dimension array to describe HEX, but slice of object cannot be specified for object that has an array type of more than one dimension, so 8-to-1 multiplexer is a must for design in part (2). Also, we should mind that instantiation(“port map”) can't be placed in “process” module while conditional statement and for loop must be.

This project deepened my understanding of digital logic design and VHDL programming. By practically designing a 16-bit synchronous counter and a scrolling display “HELLO”, I became more familiar with the working principle of digital circuits and learned how to break down complex systems into smaller, more manageable modules, such as breaking down a 16-bit counter into four 4-bit counters, and how to integrate these modules into a larger system.